CS3460: Data Structures
Homework 04: Fun with Dynamic Sequences
Total Points: 25

In this assignment, we will build a robust binary tree, balanced using randomized mechanisms, to store a dynamic sequence. We will then use this tree to solve a fun problem of ordering 100000 football teams such that each team i in the ordering defeated team $i + 1$.

1 Randomly Balanced Binary Search Trees

A randomly-built binary search tree (RBST) provides all binary search tree operations in only $O(\log n)$ time with high probability. In this problem, we will build a RBST to encode a dynamic sequence. A dynamic sequence $a_1 \dots a_n$ is a mathematical object that represents a sequence of items. The ith item in the sequence is a_i. A dynamic sequence can be encoded in a RBST, by storing some item a_i at the root, and recursively storing the sequence $a_1 \dots a_{i-1}$ as the left subtree of the root, and the sequence $a_{i+1} \dots a_n$ as the right subtree of the root. If each node in the tree is augmented with subtree sizes, then the ith item can be accessed using the `select` operation.

The file `rbst.py` contains some startup code. It contains the structure of the `Node` objects that are used in the binary search tree. Each `Node` object contains a `data` field, and each node is augmented with the `size` of its subtree. The `increment_size` and `update_size` methods allow us to increment or update this `size` field.

The main implementation of the RBST is contained in the `RBST` class. It contains instance fields `root` which is a reference to the root node of the tree. The class is currently filled with essential functions like `__len__` (which returns the size of RBST), and several public wrapper methods that are described below. Please complete the `RBST` class, so that it fully supports the following operations.

1. `insert_normal(data, rank)`: This method inserts the `data` at position `rank`. This is a simple insert routine without any balancing mechanism.
2. `inorder()`: This method performs an inorder traversal of the tree. It returns a generator that can be used to step through each individual element.
3. `split(rank)`: This method splits the tree at position specified by `rank`. It returns an an array of two RBSTs, the first being the left side of the split, and the second being the right side of the split.
4. `insert(data, rank)`: This method inserts the `data` at position `rank`. The insert routine maintains balance in a probabilistic manner. That is, it inserts the new node at the root of the tree with probability $1/n$, where n is the size of the tree including the new node. This is ideally done by splitting the tree at rank `rank-1`, and attaching the left side and right side of the split as the left and right children of the new node respectively. With probability $1 - 1/n$, the data is recursively inserted to either the left tree or the right tree depending upon the position `rank` (whether `rank` is less or greater than the rank of the root).
5. `select(rank)`: This method returns the node in the RBST at position `rank`.

An element has rank k if it is appears in position k in the inorder traversal of the tree. Since the tree encodes a sequence, the element with rank $k = 1$ and $k = n$ are the first and last elements in the sequence respectively.

Note that the above methods are public interfaces, which are wrapper methods that contain calls to corresponding private methods. These are already written and should not be modified. Please fill out places the main recursive functions.

The above class is designed with wrapper methods because of two reasons. Firstly, all the above methods are recursive, and operate on a single node which represents the tree rooted at that node, instead of a `RBST` object. Secondly, it allows the user to interact much more easily. For instance, the user can simply create and access nodes in the following manner.

```
T = RBST()       # create an instance of RBST.
T.insert(15, 10) # insert 15 at position 10.
T.select(7)      # select or access the node at position 7.
```

1.1 Testing

You are provided with a `rbst_test.py` file that tests all the methods of your RBST class. Since this class contains many methods, some of which are dependent on others, you are strongly advised to test them one at a time in order. Please do not attempt to test all the methods at the same time. You may want to comment or uncomment certain lines of code in `rbst_test.py` to test only those methods you have completely written. The program creates the same sequence stored both as an array, and as a `RBST`, and performs the same set of operations, and prints them to screen. Make sure the output printed from the array matches the output from your RBST.

2 Ordering Football Teams

Suppose n football teams, numbered $1 \dots n$, all play each other in a large tournament, where there are no ties. In file `ordering_teams.py`, you will find a method called

```
boolean did_x_beat_y(x, y)
```

which returns `True` if team x won its game against team y, `False` otherwise. Your task is to compute an ordering of the teams, say $a_1 \dots a_n$, so that team a_1 defeated team a_2, team a_2 defeated team a_3, and so on. Let's call such a sequence valid.

Note that there could be many possible valid solutions. For example, if team 1 beat team 2, team 2 beat team 3, etc., and team n beat team 1, then any "cyclic rotation" of the sequence $1, 2, 3, \dots n$ (e.g., $3, 4, 5, \dots n-1, n, 1, 2$) will be a valid solution.

The algorithm we will use to solve this problem is straightforward. It involves building up a sequence (stored in a `RBST`) to which we add teams $1 \dots n$ one by one. Suppose we have built up a valid sequence of teams $1 \dots k-1$ and we want to add team k in a position so that the entire sequence is valid. It turns out that there always exists such a position, and that we can find it quickly using our balanced `RBST`. If team k defeated the first team in our sequence, then we can just add it to the beginning. Likewise, if team k lost to the last team in the sequence, we can add it

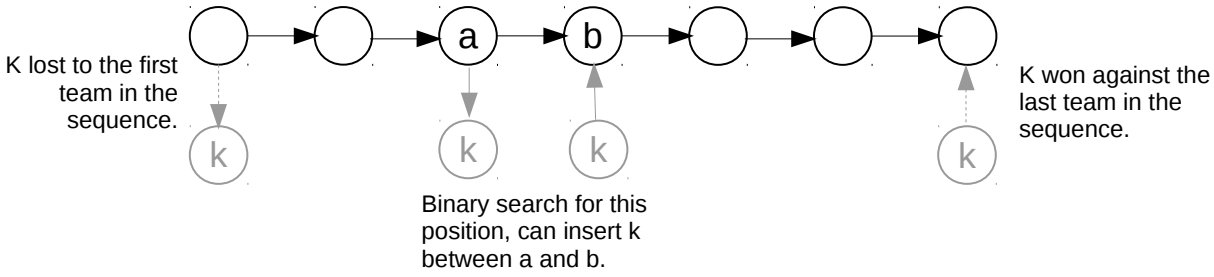


Figure 1: Inserting team k into a valid ordering of teams $1 \dots k-1$. Team k lost to the first team, and won against the last team. Therefore, we can binary search to find the position for k in the ordering. Team k can be placed between a an b, since $a \to k$ and $k \to b$.

to the end. If neither of these cases holds, then let us imagine the sequence with arrows going from the winning team to the losing team (see Figure 1). That is, $a \to b$ means that team a won against team b. Since we know that team k lost to the first team, and beat the last team, there must be some location in the ordering where there is an adjacent pair of teams (a, b) for which $a \to k$ and $k \to b$. It is thus feasible to place k between a and b. Moreover, we can binary search for such a position with only $O(\log k) \leq O(\log n)$ calls to `select`. Since each call to select takes only $O(\log n)$ time with high probability, this gives a total running time of $O(n \log^2 n)$ with high probability to solve the entire problem. Please implement the algorithm above in the method `orderTeams`. We have already provided some code to get you started. In the `main` method, we have included testing code to see if your method returns a tree encoding a valid sequence.

3 Postamble

3.1 Input and Output

The input for all programs will always be accepted from standard input (keyboard or `system.in`) and output will always be sent to standard output (monitor or `system.out`). If the input contains many lines, then you can put your input in a separate file, say `progname.in` and re-direct this input into your program by running

```
python3 progname.py < progname.in
```

You could also redirect the output of your program to another file, say `myoutput.txt`.

```
python3 progname.py < progname.in > myoutput.txt
```

If the intended output is in file `progname.out`, then you can use the `diff` utility to check your output by taking the difference of two files.

```
python3 progname.py < progname.in > myoutput.txt
diff myoutput.txt progname.out
```

If the output of the `diff` command returns nothing, this means that your output is the same as the intended output, and is judged correct for a test case. Note that your program is only correct if it is judged correct for all test cases. You are encouraged to design different test cases and test the correctness of your programs thoroughly.

Please do not include extanneous input or output as these would throw off our automated grading scripts. The input and the output should always match the format given in the description of each problem. Sample test cases are provided for you to test the formatting of the input and output, as well as to check the correctness of your program for simple test cases. Note that we will be testing with a large number of test cases.

3.2 Submission

Please submit all the individual files to `Canvas`. Please name your programs as given in the problem description, or refer to the names in the following table.

Problem Name	**File Name**
RBST	rbst.py
Ordering Teams	ordering_teams.py